

Research Note — LLD Practice Platform

On the learner problem, why existing options do not solve it, and where this product goes.

1. The learner problem

A candidate preparing for a Low-Level Design round has a clear goal and no way to practise against it. **Problems** are easy to find — parking lot, elevator, vending machine are near-universal. What they cannot get is **a signal on their own answer**.

What they do today	Why it stalls
Read a model solution	Recognition, not production. Reading a good design feels like understanding one; they cannot tell whether they'd have produced it
Watch a walkthrough	Same problem, and it anchors them to one person's answer as if it were <i>the</i> answer
Practise with a peer	Best free option, but needs scheduling, and the feedback is only as good as the peer
Paid mock interview	Genuinely useful and genuinely scarce — cost and availability make it a few reps, not a habit
Ask a general chatbot	Instant, but unanchored: fluent paragraphs with no consistent standard, no memory of the last attempt, and a strong pull toward encouragement

The gap is not content. **It is a trustworthy, repeatable verdict on work they produced themselves.**

This is why LLD is harder to self-study than DSA. A DSA attempt has a test suite — an objective, instant signal that lets a learner iterate alone at 2am. A design attempt has no such oracle. That missing feedback loop, not the difficulty of the material, is what makes design preparation stall.

2. The insight

If design has no automatic oracle, the product's job is to **manufacture the closest honest substitute** and be transparent about how far it goes. Two consequences shape everything:
A score without attribution is worthless. A learner told "6/10 on abstraction" who cannot see why learns nothing and, correctly, does not trust it. So every criterion returns **evidence quoted from the learner's own submission** plus what would raise it, and the rubric is published *before* they write — the standard has to be a target, not a verdict.

Some of the judgement is not judgement at all. Much of what makes a submission bad is structural: missing sections, no stated trade-offs, one-line answers. That is checkable deterministically — instantly, free, identically every time. Separating it from real judgement means the unreliable expensive step is never asked to do a job a word count does better, and the learner gets specific failures instead of a vague low score.
That separation is also the answer to the obvious objection, *why not just use ChatGPT?* Because a chatbot gives an ungrounded opinion that varies run to run, remembers nothing, and cannot tell you whether you improved since Tuesday.

3. Product direction

Now (this MVP). Prove the loop is worth repeating: attempt → structured submission → attributed feedback → attempt N+1, with progress visible across attempts. History is immutable, and a "focus" line carries from one attempt's feedback into the next — so the product has an opinion about what to work on *next*, not just a score for what was done.

Next, in order of value: a **class-diagram input**, so relationships are verified structurally rather than described in prose; **per-problem rubric weighting**, since a concurrency-heavy problem should not be scored like a pricing-policy one; and **calibration** against expert judgement, published — which is what converts "the model says 7" into a number a learner is entitled to trust.

Later. The defensible asset is not the problem set, which is commodity. It is the **corpus of real attempts paired with feedback and revisions** — it makes the evaluator measurably better, supports genuine benchmarking, and is the bridge to the B2B buyer: bootcamps and placement cells that must show cohort progress on design skill, which nobody can currently measure.

Two product uncertainties are deliberately unresolved, and both are wired as replaceable seams rather than guesses baked into the code: *is an LLM the right judge* (the evaluator is a port with two live implementations) and *is structured text enough* (every evaluator reads a format-neutral `DesignDocument`). The architecture is non-committal exactly where the research is, and committed where it is not — the rubric, immutable history, and the structural/judgement split are all load-bearing.

4. Open questions

- Do learners accept a machine verdict on design, or discount it regardless of attribution? (*This is the assumption the MVP exists to test.*)
- Does score improvement across attempts track real interview performance?
- Is 4 problems enough to build a habit, or does retention need breadth first?

your own prep experience, competitor teardowns — add it here with concrete numbers or quotes. I have deliberately invented none, so as written this argues from reasoning about the problem rather than evidence you collected. Even "I asked five people preparing for interviews and four said X" materially strengthens it. This block is auto-stripped from the PDF.